

DAT32-3

Machine Learning I

Linear and Logistic Models

Extending the ML workflow from regression to classification.

Albert School — 22 hours

Contents

1	From regression to classification	2
2	Logistic regression	2
2.1	The sigmoid function	2
2.2	Decision threshold	3
2.3	Training	3
2.4	Interpreting coefficients	3
3	Classification evaluation	3
3.1	AUC-ROC	3
3.2	Selecting a metric given cost asymmetry	4
4	Confusion matrix and ROC curve	4
4.1	Threshold tuning	4
5	Multi-class classification	4
6	Feature scaling	5
7	Feature engineering for linear models	5
8	Regularization applied to logistic regression	5
9	When simple models win	5
10	End-to-end classification pipeline	6

1 From regression to classification

In DAT32-1 the target variable was continuous and the model predicted a real number. In **classification** the target is a discrete label drawn from a finite set of classes. This changes the tools required in three ways:

- **Discrete output.** The model must produce a class label, not a real number.
- **Probability interpretation.** It is useful for the model to output the probability that an instance belongs to a class, not only the class itself.
- **Asymmetric error costs.** A false positive and a false negative usually carry different business costs, so a single error count is not enough to judge a classifier.

A **business problem is a classification task** when the quantity to predict is a category: whether a customer will churn, whether a transaction is fraudulent, which of several product lines a lead belongs to. When the quantity to predict is a number (revenue, demand, price), it is a regression task.

2 Logistic regression

Logistic regression is a linear model for **binary classification**, with classes labelled 0 and 1.

2.1 The sigmoid function

A linear model produces a score $z = \mathbf{w} \cdot \mathbf{x} + b$, which ranges over all real numbers. To interpret it as a probability it is passed through the **sigmoid** (logistic) function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

The sigmoid maps $(-\infty, +\infty)$ to $(0, 1)$, is increasing, and satisfies $\sigma(0) = 0.5$. The model output is

$$\hat{p} = \sigma(\mathbf{w} \cdot \mathbf{x} + b) = P(y = 1 \mid \mathbf{x}).$$

The output $\hat{p}$ is **interpreted as the probability that the instance belongs to class 1**.

2.2 Decision threshold

To turn the probability into a class label, a **decision threshold** t is chosen:

$$\hat{y} = \begin{cases} 1 & \text{if } \hat{p} \geq t, \\ 0 & \text{if } \hat{p} < t. \end{cases}$$

The default threshold is $t = 0.5$. The threshold is a free choice and can be tuned (see Section 4.1).

2.3 Training

The parameters $\mathbf{w}$ and b are fitted by minimizing the **log loss** (binary cross-entropy) over the training set of n examples:

$$J(\mathbf{w}, b) = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)].$$

This loss is convex and is minimized by gradient descent, exactly as in DAT32-1.

2.4 Interpreting coefficients

Each coefficient w_j acts on the **log-odds**: increasing feature x_j by one unit adds w_j to $\log\left(\frac{\hat{p}}{1-\hat{p}}\right)$, multiplying the odds by e^{w_j} .

3 Classification evaluation

Classification performance is summarized by counting four outcomes on a labelled test set.

Definition 1 For a binary classifier with a chosen threshold:

- **True positives** (TP): predicted 1, actually 1.
- **True negatives** (TN): predicted 0, actually 0.
- **False positives** (FP): predicted 1, actually 0.
- **False negatives** (FN): predicted 0, actually 1.

From these counts the standard metrics are defined:

$$\begin{aligned} \text{Accuracy} &= \frac{TP + TN}{TP + TN + FP + FN} \\ \text{Precision} &= \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN} \\ F_1 &= 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \end{aligned}$$

- **Accuracy** is the fraction of correct predictions. It is misleading when classes are imbalanced.
- **Precision** is the fraction of positive predictions that are correct; it penalizes false positives.
- **Recall** is the fraction of actual positives that are caught; it penalizes false negatives.
- F_1 is the harmonic mean of precision and recall, balancing the two.

3.1 AUC-ROC

The **ROC curve** plots the true positive rate (recall) against the false positive rate F_{FP+TN}^P as the decision threshold is varied across all values. The **AUC** (area under the ROC curve) summarizes the curve in a single number between 0 and 1: 0.5 is random guessing, 1 is perfect

ranking. AUC measures the classifier’s ability to rank positives above negatives, independently of any single threshold.

3.2 Selecting a metric given cost asymmetry

The metric must reflect which error is more costly:

- When **false negatives are costly** (e.g. missing a fraudulent transaction or a sick patient), prioritize **recall**.
- When **false positives are costly** (e.g. flagging a good customer as fraud), prioritize **precision**.
- When both matter and classes are imbalanced, use F_1 or **AUC**.
- Use **accuracy** only when classes are roughly balanced and errors are symmetric.

4 Confusion matrix and ROC curve

The **confusion matrix** tabulates the four counts:

	Predicted 0	Predicted 1
Actual 0	TN	FP
Actual 1	FN	TP

It is read by inspecting where the errors fall: off-diagonal entries are the two error types. In **plain language**, precision answers “when the model says yes, how often is it right?” and recall answers “of all the real yes cases, how many did the model find?”.

The ROC curve (above) is communicated as: a curve bowing toward the top-left corner is a better classifier, and the diagonal line is no better than chance.

4.1 Threshold tuning

Lowering the threshold t classifies more instances as positive, **raising recall and lowering precision**; raising t has the opposite effect. The threshold is therefore chosen to achieve the precision-recall balance demanded by the business cost asymmetry, not left at the default 0.5.

5 Multi-class classification

When there are $K > 2$ classes, binary logistic regression is extended in two ways.

- **One-vs-rest** (OvR): train K binary classifiers, the k -th separating class k from all others. At prediction time the class whose classifier gives the highest score is chosen.
- **Softmax** (multinomial logistic regression): a single model with one score z_k per class, converted to probabilities by the softmax function

$$P(y = k \mid \mathbf{x}) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}},$$

which produces a probability distribution over the K classes.

Evaluation extends to multi-class by computing precision, recall and F_1 per class and then **averaging: macro-averaging** (unweighted mean over classes, treating each class equally) or **micro-averaging** (pooling all counts, weighting by class frequency). The confusion matrix becomes $K \times K$.

6 Feature scaling

Linear and logistic models with gradient descent or regularization are sensitive to the **scale** of the features. Two methods put features on a comparable scale:

- **Standardization**: subtract the mean and divide by the standard deviation,

$$x' = \frac{x - \mu}{\sigma},$$

producing features with mean 0 and standard deviation 1.

- **Normalization** (min-max scaling): rescale to a fixed range, usually $[0, 1]$,

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}.$$

Scaling **matters** when the algorithm uses distances or gradient descent, or when regularization penalizes coefficients (features on larger scales would otherwise be penalized unequally). It is **irrelevant** for models that are invariant to feature scale. The scaler must be fitted on the training set only and then applied to the test set, to avoid data leakage.

7 Feature engineering for linear models

A linear model can only fit linear relationships in its inputs. Feature engineering adds new inputs so that a linear model can capture more structure.

- **Polynomial features**: add powers of a feature $(x^2, x^3, \dots)$ so the model can fit curved relationships.
- **Interaction terms**: add products of features $(x_1 x_2)$ so the model can capture effects that depend on two features jointly.
- **Encoding categorical variables**: convert categories into numeric features, typically by **one-hot encoding** (one binary column per category).

8 Regularization applied to logistic regression

Regularization adds a penalty on the coefficients to the log loss, exactly as in DAT32-1, controlled by a strength hyperparameter.

- **L1** (lasso) penalizes $\sum_j |w_j|$. It drives some coefficients to exactly zero, performing **feature selection**. Prefer it when many features are expected to be irrelevant.
- **L2** (ridge) penalizes $\sum_j w_j^2$. It shrinks coefficients toward zero without eliminating them, giving **coefficient stability**. Prefer it when features are correlated and the goal is stable, well-conditioned coefficients.

The regularization strength is a hyperparameter, tuned on a validation set or by cross-validation.

9 When simple models win

Logistic regression is often the right model in production, not only a baseline:

- **Interpretability**: each coefficient has a direct meaning (effect on the log-odds), which can be explained to stakeholders and audited.
- **Data efficiency**: it needs less data to fit reliably than complex models.
- **Inference speed**: prediction is a single dot product, fast and cheap to serve.

A simple model that is interpretable, cheap, and nearly as accurate is preferable to a complex one whose marginal accuracy does not justify its cost.

10 End-to-end classification pipeline

A complete classification project chains the steps above into a single reproducible scikit-learn pipeline:

1. Split the data into training and test sets.
2. Preprocess: encode categorical variables, scale numeric features (fitted on the training set only), engineer features as needed.
3. Fit a logistic regression with an appropriate regularization.
4. Choose the decision threshold from the business cost asymmetry.
5. Evaluate on the test set with the metric justified by that cost asymmetry (precision, recall, F_1 , or AUC) and inspect the confusion matrix and ROC curve.
6. Produce a **written evaluation report** that states the chosen metric and threshold and **justifies them in business terms**.